nextwork.org

Containerize a RAG API with Docker

Ahmed Tetteh

```
king@king-TFG5577XG:~/Desktop/nextwork-rag-api$ docker run -p 8001:8000 r
ag-app
2026-01-10 07:24:13,109 INFO Using model: tinyllama
2026-01-10 07:24:13,183 INFO Anonymized telemetry enabled. See
         https://docs.trychroma.com/telemetry for more information.
INFO:     Started server process [1]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
```

Ahmed Tetteh
NextWork Student

Introducing Today's Project!

In this project, I will demonstrate how to create a docker image for my RAG API and push it to container registry like Docker Hub. I'm doing this project to learn the way big tech companies like Netflix and Spotify work with applications they are developing seamlessly on any environment or system.

Key services and concepts

Services I used were AI, Docker, Docker Hub , FastAPI, RAG (Retrieval-Augmented Generation), Chroma and Ollama. Key concepts I learnt include how to package my API in a Docker container, Build Docker images from Dockerfiles, Run containerized applications and Test APIs running in containers.

Challenges and wins

This project took me approximately 3 hours. The most challenging part was accessing my containerized RAG API via the right URL path. It was most rewarding to see how containers and Docker Hub can be integrated into the developer processes to speed up work, while still ensuring security and availability.

Ahmed Tetteh
NextWork Student

Why I did this project

I did this project because knowing how to utilize docker is essential in modern day DevOps practices.

Ahmed Tetteh
NextWork Student

Setting Up the RAG API

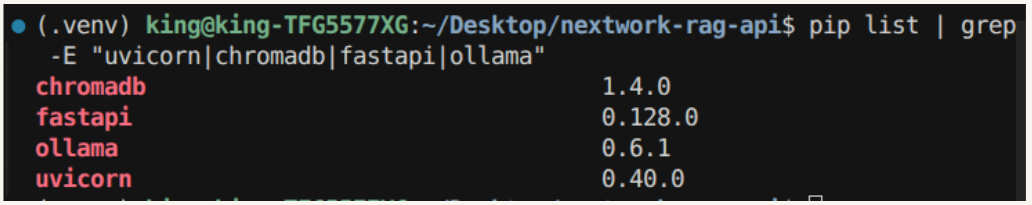
In this step, I'm setting up my RAG API's code, database and dependencies. The RAG API consits of a RAG system (AI model) and a RestAPI (web API) working seamlessly together to deliver accurate responses to prompts based on the knowledge base.

API setup and workspace

In this step, I should have already setup my virtual environment for my API to isolate all my API's dependencies from my global python packages. I will install Docker Desktop for package everything into a docker image.

Dependencies installed

The packages I installed are Fastapi, chromadb, ollama and uvicorn. FastAPI is used for building APIs. Chroma is used for storing embeddings for vector databases. Uvicorn is a server that runs FastAPI applications and listens for incoming requests and then then routes them to the correct API endpoint . Ollama is python's client for Ollama.



Local API working

I tested that my API works inputting a curl command in the terminal for my access my /query endpoint. The local API responded with "{ "answer": "Response: Kubernetes is a container orchestration platform commonly used to manage and deploy containers at scale, allowing for greater flexibility in managing container-based applications.....}". This confirms that the API works and my /query endpoint can be reachec b because it accepted my prompt, went to the database to fetch data regarding the prompt and sent everything to Tinyllama(AI model) to generate a response for me.

```
king-TFG5577XG:~/Desktop/nextwork-rag-api$ curl -X POST
"http://127.0.0.1:8000/query" -G --data-urlencode "q=Wha
t is Kubernetes?"
{"answer":"Answer: Kubernetes is a container orchestrati
on platform that is designed to make managing, deploying
, and scaling applications easier for developers and sys
tem administrators. It allows users to create and manage
 containers on a large scale, enabling organizations to
run their most demanding workloads with ease and efficie
ncy."}king@king-TFG5577XG:~/Desktop/nextwork-rag-api$
```

Ahmed Tetteh
NextWork Student

Installing Docker Desktop

Docker Desktop setup

Docker Desktop is an interactive UI tool that makes simplifies the use of docker for applications. I installed it because it makes docker issue to use by having everything preinstalled, including the Docker daemon, our main engine that runs the docker commands. Containerization will help my project by bundling all my applications dependencies, files and packages into a container that can be run on any system.

Docker verification

I verified Docker is working by running a simple command "docker run hello-world". The hello-world container proves that docker is running successfully and can create containers from docker images (templates).

```
king@king-TFG5577XG:~$ docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
17eec7bbc9d7: Pull complete
Digest: sha256:d4aaab6242e0cace87e2ec17a2ed3d779d18fbfd03042ea58f2995626396a274
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.
```

Ahmed Tetteh
NextWork Student

Creating the Dockerfile

In this step, I will be creating a Dockerfile for my RAG API. RAG stands for Retrieval Augmented Generation.

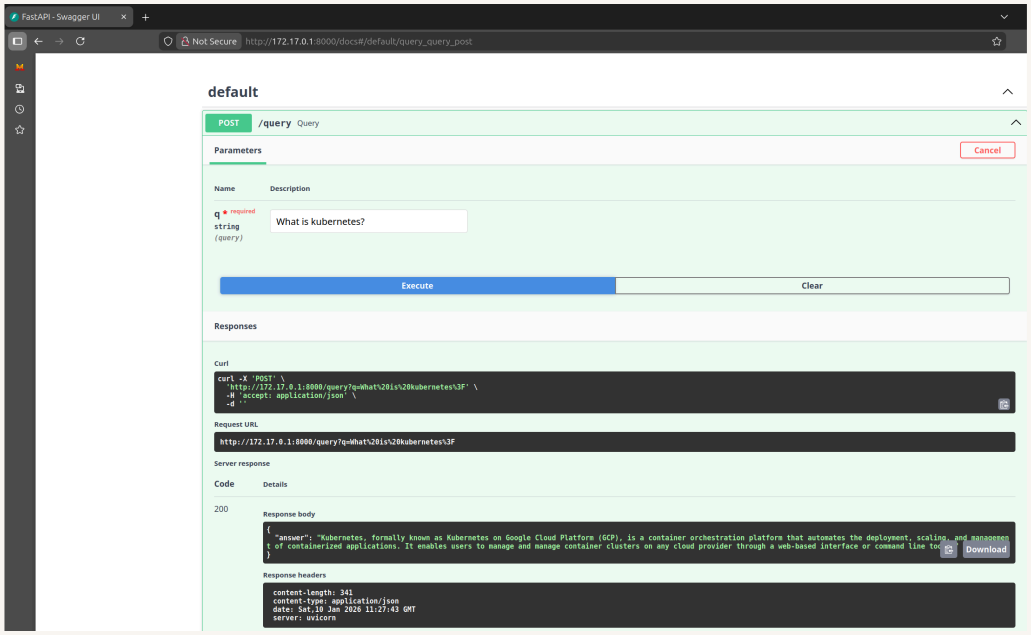
How the Dockerfile works

A Dockerfile is a text file of containing instructions that docker follows to build a docker image. The key instructions in my Dockerfile are: FROM tells Docker to download the base image python 3.11, COPY is used for copying all the code files for the app to the working directory of your conatainer, RUN installs all the Python packages for the RAG API, and CMD defines the command to run when the container starts, which launches the FastAPI server using Uvicorn and exposes it to be accessed on port 8000 of the container from any IP addresss.

Containerized API test results

Testing the API after containerization proved that my RAG API run successfully in a container. The difference between running locally and in Docker is that, locally meant it used my system's Python and packages, making it dependent on my specific environment and prone to breaking if dependencies change, but running the application in Docker meant it used a consistent Python environment inside an isolated container, including all dependencies within the image. Containerization helps because it ensures that application will be viable across any machine with Docker.

Ahmed Tetteh
NextWork Student

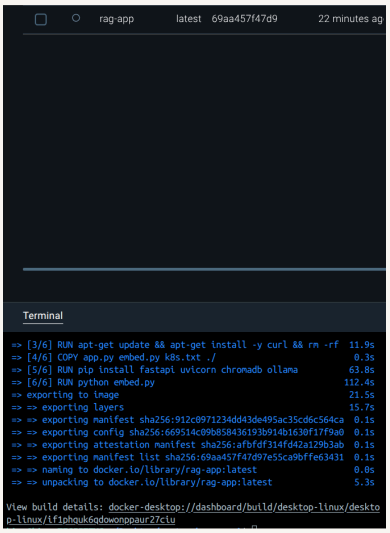


Ahmed Tetteh
NextWork Student

Building and Running the Container

Docker image build complete

Building a docker image involves creating a dockerfile containing all the instructio the docker daemon will follow to build the image. I verified my Docker image was built successfully by checking the images tab on Docker Desktop or running the command "docker images" This confirms that my API is now containerized because all the packages and packages needed for my API have all been bundled together to create a container and run on any system.



```
king@king-TFG5577XG:~/Desktop/nextwork-rag-api$ docker run -p 8001:8000 r
ag-app
2026-01-10 07:24:13,109 INFO Using model: tinyllama
2026-01-10 07:24:13,183 INFO Anonymized telemetry enabled. See
         https://docs.trychroma.com/telemetry for more information.
INFO:     Started server process [1]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
```

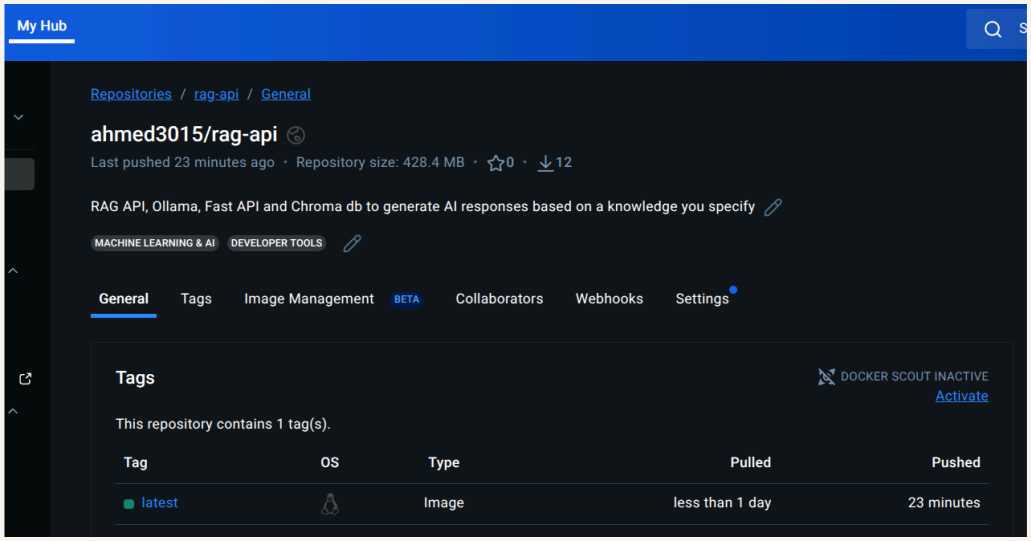
Ahmed Tetteh
NextWork Student

Pushing to Docker Hub

In this project extension, I'm pushing my docker image to Docker Hub. Docker Hub is a container registry for storing docker images and making them publicly available. I'm doing this because make my RAG API publicly available to the world and learn how developers share their works with each other.

Docker Hub push complete

I pushed to Docker Hub by using the docker push command with the new image tag docker hub can identify. Docker Hub is useful because provides a repository to store our docker images and make them either private or public. The advantage of pushing to a registry is we can backup our images on the cloud and download them again with a single command "docker pull ImageName".



Pulling from Docker Hub

Pulling an image from Docker Hub means we are downloading an image from the hub. When I ran docker pull, Docker search for the image on docker hub and downloaded it from my repository. The difference between building locally and pulling from Docker Hub is that building consists of making up all the different parts (layers) that form and image by following instructioins from a Dockerfile, while pulling from Docker Hub is simply fetching an already built image unto your local system.

```
king@king-TFG5577XG:~/Desktop/nextwork-rag-api$ docker pull ahmed3015/rag
-api
Using default tag: latest
latest: Pulling from ahmed3015/rag-api
8766df2001dd: Pull complete
01868d4b311c: Pull complete
2718fd52ab3a: Pull complete
c0789c3ed65d: Pull complete
f7f7dc91c28b: Pull complete
Digest: sha256:aee8ce62541f7e2b5f6adb605e181141fe84559f6755e54626a1a03640
651b03
Status: Downloaded newer image for ahmed3015/rag-api:latest
docker.io/ahmed3015/rag-api:latest
```

The place to learn & showcase your skills

Check out nextwork.org for more projects